Veritas Protocol — Smart-Contract Audit Report

Provenance-Aware Impermanent-Loss Protection · Uniswap v4 hook · UHI9 Hookathon · Generated 2026-06-06 (updated from 2026-06-04)

> **Scope.** Five Solidity contracts + 2 interfaces (1165 LOC) compiled with solc 0.8.26 (Foundry 1.3.5, evm `cancun`). **Result:** **81 / 81 TESTS PASS** Coverage — Lines **98.3%**, Branches **83.8%**, Functions **98.1%** (target ≥ 80%). This report maps every security property to the code location enforcing it and the test that proves it, so each row is independently verifiable via forge test --match-test <name>.

1 · COMPONENTS

CONTRACT	LOC	RESPONSIBILITY	TRUST / INVARIANTS
`VeritasOracle.sol`	223	Single signature-verification authority for off-chain DRS.	2-of-3 EIP-712 quorum · spread ≤ 5% · 10-min freshness · DRS ≤ 9500.
`VeritasRegistry.sol`	338	On-chain commitment store (fingerprints + DRS); state-transition rules; near-duplicate index.	Update timelock 1 h / 2 h · bonded disputes · inverted staleness · noisy-OR A/D combination · delegates sig-verify to Oracle.
`VeritasHook.sol`	323	v4 hook: pool gate, DRS-calibrated dynamic fee, LP Insurance Reserve.	Owner-authenticated creation · dynamic-fee-only · reserve paid to LPs only on a verified dilution event.
`VeritasRegistryCallback.sol`	65	Reactive Network destination-chain callback; receives dilution events from RSC and applies on-chain dilution increments.	Only callable by Reactive Callback Proxy (AbstractCallback.authorizedSenderOnly) · dual-hash Hamming τ=10 · skips frozen records without reverting.
`reactive/DilutionMonitorRSC.sol`	62	Reactive Smart Contract (source-chain); subscribes to Registry's NewAttestation events and triggers VeritasRegistryCallback.	Deployed on Reactive Network · forwards attestation ID via Callback event · no on-chain state mutations.
`interfaces/ (×2)`	154	IVeritasOracle, IVeritasRegistry.	—

Fee model (both DRS-driven). LP fee = `base + (max−base)·DRS/10000` (beforeSwap, override flag). Reserve skim = `2500·DRS/10000` hundredths-bip on swap output (afterSwap, ERC-6909 claims) → donated back to in-range LPs via `unlock→donate→settle`. Rationale: for a 50/50 CP pool E[IL] ∝ price variance, so DRS-as-volatility-proxy makes the fee a principled IL compensation rate. DRS = noisy-OR combination of off-chain A signal and on-chain D (dilution count), so neither path can be bribed to suppress the final score independently.

2 · TEST RESULTS BY SUITE

SUITE	TESTS	STATUS
`VeritasOracle.t.sol`	14	pass
`VeritasRegistry.t.sol`	16	pass
`VeritasHook.t.sol`	17	pass
`Integration.t.sol`	3	pass
`EdgeCases.t.sol`	21	pass
`ReactiveIntegration.t.sol`	10	pass
Total	**81**	**100%**

COVERAGE	LINES	BRANCH	FUNC
`Oracle`	97.7	100	92.3
`Registry`	99.3	79.2	100
`Hook`	97.6	75.0	100
`RegistryCallback`	100	100	100
`DilutionMonitorRSC`	90.9	—	100
Total %	**98.3**	**83.8**	**98.1**

Uncovered branches are unreachable defensive paths (e.g. low-level `TransferFailed`, the `_beforeSwap` no-config passthrough that cannot occur because every pool is gated at creation). Registry branch coverage dropped from 88.9% to 79.2% because the expanded near-duplicate and noisy-OR logic adds new conditional paths; the uncovered branches are conservative fallback edges. DilutionMonitorRSC has one uncovered branch (the `REACTIVE_IGNORE` no-op path in the `react()` callback that the Reactive VM never sends during tests).

3 · SECURITY PROPERTY → ENFORCEMENT → PROOF

PROPERTY / THREAT MITIGATED	ENFORCED IN	TEST PROVING IT
High-DRS content cannot open a pool (hard gate 0.85)	`Hook.registerPool / _beforeInitialize`	`test_RegisterPool_RejectsHighDRS · test_Demo_ThreeContentTiers`
Only the content owner can create the pool (fixes router-as-sender bug)	`Hook.registerPool (msg.sender==owner)`	`test_RegisterPool_RequiresOwner`
Pool must be a dynamic-fee pool (else fee override reverts)	`Hook.registerPool / _beforeInitialize`	`test_RegisterPool_RejectsNonDynamicFee`
Gate re-checked at the exact initialization moment	`Hook._beforeInitialize`	`test_BeforeInitialize_ReverifiesGateAtInit · _RejectsUnconfigured`
2-of-3 operator quorum; duplicate signers rejected	`Oracle.verify`	`test_Verify_RejectsBelowQuorum · _RejectsDuplicateOperator`
Unauthorized signer rejected	`Oracle.verify`	`test_Verify_RejectsUnauthorizedSigner`
DRS bounded ≤ 9500 (reject extreme oracle output)	`Oracle.verify`	`test_Verify_RejectsDRSOutOfBounds`
Cross-operator agreement: spread ≤ 5%	`Oracle.verify`	`test_Verify_RejectsSpreadTooWide`

Signature freshness window (anti-replay)	`Oracle.verify`	`test_Verify_RejectsStaleSignature · _RejectsFutureSignature`
Update timelock 1 h, escalated to 2 h on >30% jump (circuit breaker)	`Registry.updateDRS`	`test_UpdateDRS_TimelockPreventsImmediate · _LargeDeltaRequiresLongerTimelock`
Inverted staleness: stale record → conservative higher DRS, never base fee	`Registry.getEffectiveDRS`	`test_EffectiveDRS_StalenessFloorRaisesLowScore`
Disputes require a slashable bond (anti-griefing)	`Registry.disputeAttestation / resolveDispute`	`test_Dispute_RequiresBond · _ResolveDispute_RejectedSlashesBond · _UpheldRefundsBond`
A dispute freezes DRS updates & blocks pool creation	`Registry.updateDRS · Hook.registerPool`	`test_Dispute_FreezesDRSUpdates · test_FullFlow_DisputeFreezesDRS`
Attestation fee deters Sybil mass-attestation	`Registry.attest`	`test_Attest_RequiresFee`
LP Insurance Reserve accrues only when DRS > 0	`Hook._afterSwap`	`test_Reserve_AccruesOnSwap · _ZeroDRS_NoAccrual`
Reserve pays out to LPs only on a verified dilution event	`Hook.disburseReserveToLPs`	`test_Reserve_DisburseRevertsWhenNotEligible · _DisburseToLPsOnDilutionEvent`
Unlock callback callable only by the PoolManager	`Hook.unlockCallback`	`test_Hook_UnlockCallback_OnlyPoolManager`
Canonical DRS = average of signed values within spread	`Oracle.verify`	`test_Verify_ReturnsAverageWithinSpread`
Reactive callback callable only by authorized Callback Proxy (prevents spoofed dilution)	`VeritasRegistryCallback.authorizedSenderOnly`	`test_Callback_OnlyAuthorizedProxy`
Batch dilution skips frozen (disputed) records without reverting the whole batch	`VeritasRegistryCallback._tryIncrement`	`test_Callback_SkipsFrozenRecord`
Near-duplicate detection requires BOTH hashes to match (dual-hash AND logic, prevents false positives)	`Registry.getNearDuplicates`	`test_GetNearDuplicates_RequiresBothHashes`
On-chain dilution auto-raises hook fee without oracle update (bribe-resistant floor)	`Registry.getCurrentDRS / Hook._beforeSwap`	`test_OnChainDilution_RaisesHookFee`
Effective D = max(off-chain oracle, on-chain count): bribed-low oracle cannot hide on-chain evidence and vice-versa	`Registry.getEffectiveD`	`test_EffectiveD_MaxOfOnchainAndOffchain`

4 · REPRODUCE THE AUDIT YOURSELF

```
cd contracts
forge test -vv                 # 81 passing
forge coverage --no-match-coverage "(test|lib)"   # the table in §2
forge test --match-test test_Demo_ThreeContentTiers -vvv   # A opens / B higher fee / C gated
forge test --match-test test_Callback_DilutesNearDuplicatesOnChain -vvv   # RSC callback flow
```

Dependencies are pinned to a single consistent set (OpenZeppelin `uniswap-hooks` bundle + `reactive-lib`); `BaseHook` resolves to `lib/uniswap-hooks/src/base/BaseHook.sol`. Deployment order (Oracle → Registry → Hook → VeritasRegistryCallback → DilutionMonitorRSC, hook address CREATE2-mined for permission bits) is in `script/Deploy.s.sol` and `script/DeployCallback.s.sol`.

5 · HONEST LIMITATIONS (IN SCOPE FOR THE AUDIT)

- **Oracle centralization.** 2-of-3 is multi-sig, not trustless. Real independence requires each operator to run a *different* detector ensemble off-chain — enforced operationally, not on-chain.
- **On-chain near-duplicate index is bounded.** `getNearDuplicates` performs an O(n) scan over all attested fingerprints on-chain; this is viable at early corpus sizes but will require pagination or a Merkle-tree index at scale.
- **DRS is a lagging, ex-ante signal** — it prices risk at entry and on updates, not in real time. Acknowledged, not hidden.
- **Reactive Network trust assumption.** The Callback Proxy is a privileged actor in the Reactive Network. `authorizedSenderOnly` ensures only the proxy can call `onDilution`, but the proxy itself is a trusted third party — on-chain dilution counts are only as reliable as the Reactive Network's liveness and integrity.
- **Deployed to Unichain Sepolia.** Oracle, Registry, Hook, VeritasRegistryCallback, and two v4 pools are live on Unichain Sepolia. DilutionMonitorRSC is deployed on the Reactive Network (Kopli testnet). Mainnet deployment is a separate phase.
- **Not a formal audit.** This is a self-authored, test-backed engineering report — no third-party review or formal verification performed.

Veritas Protocol · contracts/ package · solc 0.8.26 · Foundry forge 1.3.5-stable · 81 tests, 0 failures · report covers code as of 2026-06-06 (original 2026-06-04; updated to reflect VeritasRegistryCallback + DilutionMonitorRSC + ReactiveIntegration suite + Unichain Sepolia deployment). Each claim in §3 is checkable in isolation; discrepancies should be treated as audit findings.